SPACEBOX

New game mechanics

Ideas that build on what the game already has, and a prioritized roadmap

31 ideas in 9 areas: survival, exploration, automation, ships,
threats, economy, multiplayer, endgame and world events
Based on game version 0.1.5 (spec.md, ships-design.md, tutorial-design.md, Default GameSet)

Design proposal · October 2026

Contents

About this document 3

Overview of all ideas 4

Impact versus effort 5

S Survival pressure 6

X Exploration of the belt 8

A Automation and logistics 10

SH Ship gameplay 12

T Threats and enemies 14

E Economy and trade 16

M Multiplayer: co-op and PvP 17

G Endgame goals 19

EV World events 20

Prioritized roadmap 22

About this document

Spacebox already has a deep set of systems: voxel asteroids with ores and cavities, machines and power grids, pipes, oxygen rooms, gravity generators, DNA respawn, parasites with real AI, player-built ships with thrusters, cannons, consoles and collisions, and a server-authoritative multiplayer with claims, rules and a web dashboard. What it lacks is **reasons**: reasons to leave the first asteroid, to keep a base running, to fly far, and to play together over weeks.

Every idea below adds such a reason by **reusing existing systems** instead of inventing parallel ones. Each card names the systems it builds on and gives engine-specific notes on performance, networking and the save format, following the project rules (data-driven GameSets, server authority, optional save keys, no per-frame allocations, tutorials in sync).

Ideas that overlap with `Spacebox/TODO.txt`, the goal list at the top of `DONE.txt` or the planned WP10 in `ships-design.md` are marked with an **overlap** note that says what new value the card adds.

How to read the cards

Effort	S: up to 1 week · M: 1-3 weeks · L: 3-6 weeks · XL: more than 6 weeks (one developer working with an agent, tests included)
Priority	P1 next versions, high leverage P2 mid-term P3 later or optional
Impact	1-5 dots: estimated effect on the core loop and on retention
Builds on	Existing classes and systems the idea reuses (names as in the code base)

Design principles for every new mechanic

Data first	Every mechanic is driven by a new JSON section with defaults (`Fuel`, `Thruster.Fuel`, `Radiation`, `contracts.json`, `events.json`). The `Default` GameSet keeps loading and mods can tune or disable the feature.
Server-authoritative	Anything that changes the world or inventories is decided on the server and synced through `BlockStateNetwork`, an `INetMessage` or a `NetworkVariable`. Visual-only work stays on clients (survey pulse, comfort score).
Toggle with a rule	Every pressure mechanic gets a `ServerRules` switch (via `ServerRulesService`): `decompression`, `fuel`, `mood_effects`, `world_events`, `sieges`. Single player keeps them in `rules.json`.
Compatible saves	New NBT and JSON keys are optional with defaults. Only renamed or removed ids need a `WorldUpgradeStep`. None of the ideas needs to change the `Block` bit layout.
Hot paths stay clean	New ticking systems are per entity (like `DoorSystem` / `OxygenSystem`), run at 1-4 Hz on the server, allocate nothing per frame and use workers plus `MainThreadDispatcher` for scans.
Tutorial in sync	Every mechanic a beginner needs (fuel, survey pulse, crews) gets a tutorial step in the same change, and `DefaultTutorialContentTests` stay green.

Overview of all ideas

ID	Idea	Area	Effort	Prio	Impact
S1	Fuel-burning generators and battery banks	Survival pressure	M	P1	●●●●●
S2	Hull breaches and decompression	Survival pressure	M	P2	●●●●○
S3	Radiation	Survival pressure	M	P2	●●●○○
S4	Comfort and mood	Survival pressure	S-M	P2	●●●○○
X1	Derelicts and structures OVERLAP	Exploration of the belt	L	P1	●●●●●
X2	Resource biomes OVERLAP	Exploration of the belt	M	P1	●●●●○
X3	Survey scanner pulse	Exploration of the belt	S-M	P1	●●●●○
A1	Auto-miner OVERLAP	Automation and logistics	M	P1	●●●●●
A2	Signal channels: sensors and actuators	Automation and logistics	L	P2	●●●●○
A3	Logistics upgrades: sorter, stock signs, storage terminal OVERLAP	Automation and logistics	M	P2	●●●○○
A4	Cargo launcher (mass driver)	Automation and logistics	L	P3	●●●○○
SH1	Docking connectors and landing clamps OVERLAP	Ship gameplay	L	P1	●●●●●
SH2	Propellant and fuel tanks	Ship gameplay	M	P2	●●●●○
SH3	Navigation computer: autopilot and cruise OVERLAP	Ship gameplay	M	P1	●●●●○
SH4	Ship damage model and splitting OVERLAP	Ship gameplay	XL	P2	●●●●○
SH5	Tractor beam and salvage cutter	Ship gameplay	M	P3	●●●○○
T1	Parasite nests and infestation	Threats and enemies	M	P1	●●●●●
T2	Automated defense turrets	Threats and enemies	M	P2	●●●●○
T3	Scavenger drones and pirate ships	Threats and enemies	XL	P2	●●●●●
E1	Contracts board (reuses the tutorial engine)	Economy and trade	M	P1	●●●●●
E2	Trade outposts and Spacer merchants	Economy and trade	L	P2	●●●●○
E3	Research by analysis	Economy and trade	M	P2	●●●●○
M1	Crews	Multiplayer: co-op and PvP	M	P1	●●●●○
M2	Claim sieges (PvP servers)	Multiplayer: co-op and PvP	L	P3	●●●○○
M3	Leaderboards and server goals	Multiplayer: co-op and PvP	S	P2	●●●○○
G1	Belt Gate megaproject	Endgame goals	L	P2	●●●●●
G2	Leviathan hive (boss)	Endgame goals	L	P3	●●●●○
EV1	World event director	World events	M	P2	●●●●○
EV2	Meteor showers	World events	M	P2	●●●●○
EV3	Solar flares OVERLAP	World events	S	P2	●●●○○
EV4	Distress signals and comets OVERLAP	World events	M	P3	●●●○○

OVERLAP = touches an item in TODO.txt, the DONE.txt goal list or ships-design.md WP10; the card says what it adds.

Impact versus effort

Ideas in the top-left area give the most value for the work. Phase 1 of the roadmap takes most of them; the XL items on the right are scheduled once their foundations exist.



- **S** Survival pressure
- **X** Exploration of the belt
- **A** Automation and logistics
- **SH** Ship gameplay
- **T** Threats and enemies
- **E** Economy and trade
- **M** Multiplayer: co-op and PvP
- **G** Endgame goals
- **EV** World events

Best value for the work

#	Idea	Impact	Effort	Value
1	EV3 Solar flares	●●●○○	S	3.0
2	M3 Leaderboards and server goals	●●●○○	S	3.0
3	X3 Survey scanner pulse	●●●●○	S-M	2.7
4	A1 Auto-miner	●●●●●	M	2.5
5	E1 Contracts board (reuses the tutorial engine)	●●●●●	M	2.5
6	S1 Fuel-burning generators and battery banks	●●●●●	M	2.5
7	T1 Parasite nests and infestation	●●●●●	M	2.5
8	S4 Comfort and mood	●●●○○	S-M	2.0
9	E3 Research by analysis	●●●●○	M	2.0
10	EV1 World event director	●●●●○	M	2.0

Value = impact divided by effort weight (S 1, S-M 1.5, M 2, L 4, XL 6). It is only a first sort: dependencies and fun matter too, see the roadmap.

S Survival pressure

Today survival is mostly oxygen, suit power and parasites. These ideas add slow, readable pressure that rewards good base design instead of punishing the player at random. (4 ideas: S1, S2, S3, S4)

S1 Fuel-burning generators and battery banks

Effort M **Priority** P1 **Impact** ●●●●●

WHAT IT IS

Cobalt and nuclear power blocks make power for ever once they are built. Give generators an optional JSON **Fuel** section: a fuel slot (for example `cobalt_ingot` or `energy_cell` for cobalt blocks, `uranium_rods` for nuclear blocks), a burn time per item, and output only while fuel is left. Add a **battery** block type that charges from surplus power and discharges when consumption is higher than generation. A power readout on machine windows shows the net balance, the stored energy and the time until empty. Solar panels can follow once the world has a sun direction.

BUILDS ON

`ElectricNetworkManager` (the rule generation >= consumption gives `IsActive`), `GeneratorBlockData` (already reads `Power` and `Explosive`), the container system (`IContainerBlock`, `ContainerServer` snapshots), pipes (a `FuelEndpoint` adapter registered with `ItemTransferEndpoints.Register`), and the existing `uranium_rods` / `energy_cell` crafting chains.

PLAYER EXPERIENCE

Power becomes an ongoing loop: mine, refine, feed the reactor. Automating fuel with pipes feels earned. A blackout matters because the oxygen generator and the DNA reconstructor stop too.

RISKS

This changes the build-once balance. Old worlds must not lose power suddenly: fuel applies only to GameSets that define it, and the upgrade fills every generator with a grace buffer. Batteries bend the all-or-nothing network rule, so a discharging battery should simply count as generation.

TECHNICAL NOTES (performance, network, save format)

Battery charge and fuel burn must be decided on the server and synced as block state (`INetworkStateBlock`, like the gravity generator). Clients compute electricity themselves today, and a client-side burn would drift. Tick each network at 1-2 Hz through `TickTaskManager` without allocations. Saves: `battery_data` { charge } and `generator_fuel` { slot, burn_left } through an `IBlockSerializer`. Both are optional keys, so no upgrade step is needed. The tutorial's *Power* step must teach fuelling (keep `DefaultTutorialContentTests` green).

S2 Hull breaches and decompression

Effort M **Priority** P2 **Impact** ●●●●○

WHAT IT IS

When a block next to a pressurised room is destroyed (by a shot, an explosion, a cannon or a collision), the air does not just vanish. For 1-3 s it vents through the hole: players, drops and physics bodies are pulled toward it, a hiss and a particle stream play, and doors marked as **bulkheads** close automatically. Add an emergency sealant item that places a temporary foam block in the hole.

BUILDS ON

`OxygenSystem` / `AirFloodFill` (leaks are already detected), `AirMasks` (the difference between the old and new mask gives the lost cells), the push code in `ExplosionForces`, `DoorSystem` (auto-close), `Oxygen.OnBlockChanged`, `AirZoneSounds`, particles.

PLAYER EXPERIENCE

Dramatic, physical moments. Airlocks and compartments become worth building, and ship combat has consequences beyond missing blocks.

RISKS

A flood fill runs on every breach, but buffers are pooled and the radius is capped at 32. Getting sucked out again and again is annoying, so the force is capped, applies in survival only, and can be turned off with a `decompression` server rule.

TECHNICAL NOTES (performance, network, save format)

Breach = previous air mask minus the new mask on the same entity: lost cell count and centroid, plus the removed cell as the hole. The server sends `DecompressionMessage` (entity, hole cell, strength). Each client pushes only its own player (the explosion model) and the server moves drops and bodies. Nothing is saved. JSON: `Door.AutoCloseOnLeak` (default false).

S3 Radiation

Effort M **Priority** P2 **Impact** ●●●○○

WHAT IT IS

Uranium ore, uranium rods and nuclear power blocks give off radiation. In survival a **dose** builds up and lowers max health until an anti-rad consumable or time clears it. Dense blocks (heavy metals, titanium) shield. A Geiger crackle follows the dose rate, and scanner equipment shows the level.

BUILDS ON

The `GravityField.Sample` pattern (sources registered through `IEntityBlockListener`, then summed) becomes a `RadiationField`. Also `StatsData` / health, `EquipmentStatsBinder` (armor protection like `ExplosionProtection`), block `Mass` as shielding, `SuffocationDamage` as the model for damage over time, consumables.

PLAYER EXPERIENCE

Nuclear power and uranium mining get a cost, and heavy armor and shielded reactor rooms get a reason to exist. The Geiger sound adds tension.

RISKS

Invisible damage feels unfair, so it needs strong audio and visual feedback and a slow dose. Uranium ore must stay weak so normal mining stays safe.

TECHNICAL NOTES (performance, network, save format)

Sample 4 times per second for the local player, using the N nearest sources only. Shielding uses a `VoxelLine` DDA that sums block mass, cached per source/player pair until the chunk's `DataVersion` changes. Runs on the client for the player's own body (like fall damage). Saves: `Radiation` in the player JSON (default 0) and a JSON `Radiation { Strength, Range }` on blocks (default none).

S4 Comfort and mood

Effort S-M **Priority** P2 **Impact** ●●●○○

WHAT IT IS

`Mood` already exists (a component, a HUD bar, timed events and mood sounds) but has almost no gameplay. Turn it into **comfort**: a sealed room gets a score from lights, windows, pictures and signs, gravity, and seat or bed blocks. High mood gives faster health regeneration and suit power. Low mood (long periods in darkness, without air, or near parasites) slightly slows drilling. The 30 craftable pictures finally get a purpose.

BUILDS ON

`Mood` / `TimedMoodEvent`, `SealedRoomFill` / `GravityRoom` (room detection), `World.IsBreathable`, picture frames and signs, `LightManager`, `PlayerStatistics`.

PLAYER EXPERIENCE

It rewards building a home instead of a box. It is gentle and never punishing.

RISKS

Players might spam decor, so each decor type has diminishing returns. It must stay optional (a `mood_effects` rule).

TECHNICAL NOTES (performance, network, save format)
Score a room only when the player enters it or the room's chunks change (`DataVersion`), on a worker over the room's cell list. There is no per-frame flood fill. The score stays local to each player, so nothing is sent over the network. Saves: `Mood` in the player JSON (default 50).

X Exploration of the belt

Sectors hold 1250-2500 asteroids, but most of them look and play alike and the structure files are empty. These ideas give the player reasons to leave the first asteroid. (3 ideas: X1, X2, X3)

X1 Derelicts and structures

Effort L **Priority** P1 **Impact** ●●●●●

OVERLAP TODO.txt "generation"; DONE.txt header "better generation (different objects and resources)". Adds a concrete design for the empty structure files.

WHAT IT IS
Fill the empty `abandoned_ship`, `alien_base` and `ruins` definitions. A structure is a prefab `.entity` file built in creative, plus JSON: biome weights, count per sector, loot tables per storage, a damage pass (blocks randomly removed or cracked), optional mobs (a parasite nest) and optional sign texts (logs and lore). Derelict ships contain real ship blocks, so the player can repair them with the repair tool, power them and fly them away.

BUILDS ON
Entity files are position-independent (any `.entity` can be copied into any sector), `NotGeneratedEntity` placeholders, `Generation/Loot` (`chest`, `alien-chest` already exist), the `Structures` list in `generator.json` (`GenerationLoader.LoadStructures` is commented out today), the repair tool and crack stages, signs, `ShipBody` (any entity with ship blocks becomes a ship), `EntityProtection` for story sites.

PLAYER EXPERIENCE
This is the biggest missing piece of exploration: something to find between the asteroids. A derelict can be an early ship before the full tech, and signs and pictures carry environmental storytelling.

RISKS
Prefabs cost authoring time. Loot balance matters (no cheap titanium). Saving every structure in full would bloat worlds, so structures should regenerate like asteroids.

TECHNICAL NOTES (performance, network, save format)
A `StructureEntity` (or a flag on `SpaceEntity`) with `SavesOnlyChangedChunks` and NBT keys `structure_id` + seed (with defaults). Prefab chunks are decoded once and cached. The damage pass is deterministic from the seed, so clients build untouched structures themselves and nothing goes over the network, as with asteroids. Loot is rolled on the server at first open (as the Spacer does). Test: prefab placement is deterministic and stays inside the 16-chunk limit.

X2 Resource biomes

Effort M **Priority** P1 **Impact** ●●●●○

OVERLAP DONE.txt header "better generation (different objects and resources)".

WHAT IT IS

There are two biomes today (`asteroid_field`, `dense_cluster`). Make biomes the reason to travel by putting thulium, obscurium and uranium mostly in specific biomes farther from the sector centre: **Ice Ring** (icy and methane), **Iron Graveyard** (heavy metal, many derelicts), **Obscurium Rift** (dark, dense fog, rare asteroids, more parasites), **Radiant Cluster** (uranium, radiation). Each biome sets its fog colour and density, a sky tint, an ambient sound and mob spawn rules.

BUILDS ON

`BiomeGenerator` / `BiomesMap`, `MinDistanceFromCenter` / `MaxDistanceFromCenter` (already in biome JSON), asteroid presets and `AsteroidOreGenerator`, `fog.glsl` with the fog density in `GlobalUniforms`, mob `Spawn` rules, and the biome map preview in the generation editor.

PLAYER EXPERIENCE

Players build a mental map of the belt. Ships and fuel get destinations, and radar and waypoints become important.

RISKS

Untouched asteroids in existing worlds would regenerate differently if the biome map changes. Keep the existing biome ids and only add biomes in new distance bands, or version-gate them through `generator.json` `Version`.

TECHNICAL NOTES (performance, network, save format)

Look up the biome at the camera once per second and blend fog and ambient sound (no per-frame allocations). Everything is JSON with defaults. Never break the `Default` GameSet: the new biomes ship in Default but only beyond the current spawn band.

X3 Survey scanner pulse

Effort S-M **Priority** P1 **Impact** ●●●●○

WHAT IT IS

Scanner equipment (basic, iron, titanium) gets an active pulse. Hold a key, and after 1 s a sonar wave expands and outlines ore clusters within range through the rock for a few seconds, coloured by ore. It also leaves a temporary survey tag on the asteroid with the estimated content. The analyzer block stays the tool for exact numbers.

BUILDS ON

Radar equipment settings (`Range`, `ShowMass`), `AnalyzerScanner` (mass and mineral estimates), `TagManager` / waypoints, the deterministic `AsteroidOreGenerator`, world lines from `ShipOutlineRenderer` / `ClaimVisuals`.

PLAYER EXPERIENCE

Mining becomes targeted instead of digging blind. It is a satisfying action with sound and is also a perfect tutorial step.

RISKS

X-ray vision can make exploration trivial, so the pulse has a cooldown and a power cost, shows clusters rather than single blocks, and its range depends on the tier.

TECHNICAL NOTES (performance, network, save format)

Scan on a worker over the palettes of loaded chunks (chunks whose palette has no ore are skipped, which makes it cheap), merge results into cluster boxes and return them through `MainThreadDispatcher`. Draw at most 64 boxes in one line batch. Runs only on the client, with no save and no network traffic. Needs an input action with a default key and a tutorial step.

A Automation and logistics

Pipes, machines and power grids are in place. The missing parts are production that runs without the player, and control over that production. (4 ideas: A1, A2, A3, A4)

A1 Auto-miner

Effort M **Priority** P1 **Impact** ●●●●●

OVERLAP Author goal "automation" (DONE.txt header).

WHAT IT IS

A multi-cell **miner** block placed against asteroid rock. It drills the blocks in front of it in a tunnel or layer pattern, at a speed and tier from JSON (like drill items), uses power, and pushes the drops into an internal buffer that pipes empty. It is upgraded with the existing aluminium, iron and titanium drill bits.

BUILDS ON

Drill tiers and `PowerToDrill`, `BlockDestroyer.Destroy` (drops, stats, network), `IItemTransferEndpoint` and pipes, `ElectricNetworkManager`, `IReservedAreaBlockData` for the drill head, `DropManager`. Parasites hear the noise through `HearingRange`.

PLAYER EXPERIENCE

This is the *automation* goal: a base that produces while the player explores, linking pipes, furnace and storage into real factories.

RISKS

It could be a griefing tool in multiplayer, so it respects `ProtectionRules` and claims. Re-meshing has a cost, so each miner is limited to one block per second and uses the chunk re-mesh budget.

TECHNICAL NOTES (performance, network, save format)

A server-only `TickTask` (like `DisassemblerBlock`). Target cells are found in the entity space of the touched asteroid (`Sector.Raycast` when the miner is on another entity). Removals are broadcast as normal block edits through `WorldEditNetwork`, and `chunk.IsModified` must be set. Saves: `miner_data` (progress, pattern cursor, buffer slots) with defaults.

A2 Signal channels: sensors and actuators

Effort L **Priority** P2 **Impact** ●●●●○

WHAT IT IS

A light logic layer per entity that reuses the colour groups players already know from ship consoles. **Sensors:** button, lever, proximity plate, storage level, power level, air present, timer. **Actuators:** doors, gravity generators, pipes (on/off), lights, oxygen generator, and thrusters and cannons (already console-driven). Each block gets a channel colour, set with the binding tool. A channel is on when any of its sensors is on (OR), and an invert flag gives NOT.

BUILDS ON

`ConsoleBindings` / `ShipControlSystem.ApplyCommand` (actuators are already `IShipParts`), `ShipBindingTool` and `ShipOutlineRenderer` (colour outlines), `DoorNetwork`, `GravityConfigMessage`, `IsActive` power gating, `BlockStateNetwork`.

PLAYER EXPERIENCE

Self-cycling airlocks, lights that switch on when someone enters, automatic shutdown on low power, factory control. It is an engineers' playground without a programming language.

RISKS

Complexity creep (this must not become Redstone). Loops are avoided by evaluating once per tick with no chaining inside a tick.

TECHNICAL NOTES (performance, network, save format)

A per-entity `SignalSystem` (like `DoorSystem`) holding a 16-bit channel mask. Channel assignments live in a serializer map (`signal_data`: cell, channel, invert) and not in the `Block` bits, so the hot struct stays untouched. The server evaluates the channels and sends one `ushort` per entity only when it changes. Old saves have no data, which means no channels.

A3 Logistics upgrades: sorter, stock signs, storage terminal

Effort M **Priority** P2 **Impact** ●●●○○

OVERLAP Related to TODO.txt "inventory refactoring".

WHAT IT IS

A **sorter pipe** (several filter slots, routes items to sides), overflow and priority outputs, a sign token `<count=item>` that shows the stock of the attached crate, and a **storage terminal** that lists every crate on the entity and pulls items from any of them.

BUILDS ON

`PipeBlock` (filter slot, input/output sides, round robin), `StorageEndpoint`, `SignMarkup` tokens and `SignTextBaker`, `ContainerServer`, the crafting window (it could take ingredients from the terminal).

PLAYER EXPERIENCE

Less inventory juggling, and bases look alive because their signs show stock.

RISKS

A terminal over hundreds of crates is expensive, so totals are cached per entity and refreshed only when storages change.

TECHNICAL NOTES (performance, network, save format)

The sorter is a `PipeBlockData` variant with a filter list (an extension of `pipe_data`). Stock signs are resolved on the server and pushed as block state, so clients never scan storages. Optional keys only.

A4 Cargo launcher (mass driver)

Effort L **Priority** P3 **Impact** ●●●○○

WHAT IT IS

A launcher block fires a sealed cargo pod toward a paired receiver beacon on another entity. The receiver catches pods within a radius and unloads them into pipes. Pods can be shot down, lost or stolen.

BUILDS ON

`SphereBody` / `PhysicsWorld`, the drop replication model (`DropManager.Replicator`), tags, explosion push, pipes.

PLAYER EXPERIENCE

Logistics at belt scale, and emergent piracy in multiplayer.

RISKS

Pods could be lost in unloaded sectors, and generic physics bodies are not synced over the network yet.

TECHNICAL NOTES (performance, network, save format)

Pods are server-owned network objects, as drops are (spawn and remove messages, local simulation). With no gravity in open space the path is a straight line, so arrival can be computed on the server even when the sector is unloaded. Saved in the sector data (`cargo_pods`, optional).

SH Ship gameplay

Ships fly, shoot and collide (WP1-WP9 of ships-design.md are done). These ideas turn ships from a feature into a loop: fuel, travel, docking and damage. (5 ideas: SH1, SH2, SH3, SH4, SH5)

SH1 Docking connectors and landing clamps

Effort L **Priority** P1 **Impact** ●●●●●

OVERLAP Planned in ships-design.md WP10. Adds network bridging and server performance.

WHAT IT IS

A **connector** pairs with another connector face to face. The ships lock together and are piloted as one, and power, walkable passage and pipes connect across the joint. A **landing clamp** locks a ship to an asteroid, making it static: no physics cost and nothing can push it.

BUILDS ON

`ShipBody`, `SpaceEntity.SetPose`, the per-entity `ElectricNetworkManager`, `ItemTransferEndpoints`, `ProtectionRules.CanUseEntity`, and the pair handling in `ShipCollider`.

PLAYER EXPERIENCE

Motherships and shuttles, refuelling at a home base, parked fleets.

RISKS

A compound rigid body is hard (combined inertia, collisions inside the pair). The first version should attach the docked child kinematically to the parent's pose, so only the parent is simulated.

TECHNICAL NOTES (performance, network, save format)

Both entities save the dock relation (`dock_parent` id plus relative pose, default none). `ShipStateMessage` carries only the parent, and clients derive the child's pose. Electric bridge: a virtual edge between the two networks so their sums are evaluated together. Clamped ships skip `ShipCollider` entirely, a real CPU saving on large servers.

SH2 Propellant and fuel tanks

Effort M **Priority** P2 **Impact** ●●●●○

WHAT IT IS

Chemical thrusters burn propellant from tank blocks, using `liquid_hydrogen` and `compressed_methane`, which can already be crafted but are hardly used. The ion thruster needs only power but is weak. Tanks are containers filled by pipes like any storage. The console HUD shows the fuel left and the estimated range.

BUILDS ON

`ThrusterBlock` (`Force`, `Power`), `AutoFlight`, `IContainerBlock`, pipes, the ice to water to hydrogen chains in `recipes.json`, ice biomes (X2).

PLAYER EXPERIENCE

Trips need planning, ice asteroids become strategic, refineries get a purpose, and co-op gets a stranded-ship rescue loop.

RISKS

Being stranded is frustrating, so the ion thruster is the fallback, fuel can be disabled with a server rule, and creative ignores fuel.

TECHNICAL NOTES (performance, network, save format)

Consumption is computed on the server per tick for each firing thruster group, aggregated per entity. The tank fill level is block state for the HUD, and the fuel itself sits in container slots (existing serialization). Optional JSON `Thruster.Fuel { Item, PerSecond }` with default none, so existing content does not change. The *Thrusters* and *Fly!* tutorial steps need a fuel task if Default uses it.

SH3 Navigation computer: autopilot and cruise

Effort M **Priority** P1 **Impact** ●●●●○

OVERLAP Autopilot is planned in ships-design.md WP10. Cruise mode is new.

WHAT IT IS

A navigation block lets the pilot pick a waypoint or tag and engage the autopilot: `AutoFlight` turns the ship and accelerates, brakes before arrival, and avoids asteroids with ray probes. **Cruise:** when no entity is within N blocks, `MaxSpeed` rises (for example from 20 to 120) with a speed-streak effect, and drops back near obstacles.

BUILDS ON

`AutoFlight` (PD controller), `ShipPhysicsSettings.MaxSpeed`, `WaypointService` / `TagManager`, `Sector.Raycast`, `EntitySectorTracker`, `WorldObservers` (server streaming).

PLAYER EXPERIENCE

A sector is 8192 blocks wide. At 20 blocks/s crossing one takes about 7 minutes, so cruise is what makes distant biomes (X2) reachable. Pirate AI (T3) needs the same autopilot.

RISKS

Streaming may not keep up at high speed, so the cruise speed is limited by feedback from `StreamingBudget` and the server's generation queue. Watch float precision (`RenderSpace`) too.

TECHNICAL NOTES (performance, network, save format)

The autopilot runs where the ship is simulated (the server), and the target is saved in the console state (`nav_data`, optional). The network protocol does not change (`ShipStateMessage`). Sector placeholders are prefetched along the velocity vector.

SH4 Ship damage model and splitting

Effort XL **Priority** P2 **Impact** ●●●●○

OVERLAP Planned in ships-design.md WP10. Adds a critical-systems HUD and repair integration.

WHAT IT IS

Collisions damage blocks in proportion to the impact energy. Cannons can disable subsystems: an unpowered console, or broken thrusters that pull the flight to one side. When a ship is cut in two, the disconnected part becomes its own entity and keeps its velocity. The console shows a critical-systems view that highlights damaged parts.

BUILDS ON

`ShipContactSolver` impulses, `Chunk.DamageBlock` / `BlockDamage.Apply` and crack visuals, connectivity through `AirFloodFill`-style pooled buffers, `EntityCreationNetwork`, `ShipOutlineRenderer`, the repair tool.

PLAYER EXPERIENCE

Ship combat gets weight, ramming becomes possible, and wrecks can be salvaged.

RISKS

Splitting large entities is expensive and causes network bursts. Asteroids have infinite mass, so hitting rock could wreck ships too easily, which calls for a damage threshold and a server rule.

TECHNICAL NOTES (performance, network, save format)

After removals on dynamic entities, a connectivity check is queued on a worker (debounced 0.5 s). The server creates the new entity, moves cells raw (`ClearCellSilently` / `PlaceCellRaw`, as doors do) and broadcasts `EntityCreatedMessage` plus a bulk edit. Each part saves as its own entity file, so the save format does not change.

SH5 Tractor beam and salvage cutter

Effort M **Priority** P3 **Impact** ●●●○○

WHAT IT IS

A tractor block or handheld tool pulls drops, pods and small entities below a mass limit, and can tow derelicts. A salvage cutter takes blocks of unclaimed derelicts apart into components faster than mining them.

BUILDS ON

The `SphereBody` push code, `ShipBody` forces applied at points, disassembly recipes, the repair tool's interaction pattern.

PLAYER EXPERIENCE

Cleaner salvage gameplay and collecting loot after fights.

RISKS

Towing ships with players aboard; it must respect claims.

TECHNICAL NOTES (performance, network, save format)

The force is applied on the server to the target's `ShipBody` (no new sync). The cutter uses `BlockDestroyer` with disassembly output.

T Threats and enemies

The mineral parasite is the only enemy. These threats build on the mob, cannon and AI systems that already exist. (3 ideas: T1, T2, T3)

T1 Parasite nests and infestation

Effort M **Priority** P1 **Impact** ●●●●●

WHAT IT IS

Parasite **hives** (organic blocks) generate inside some asteroids, more often in the Rift biome. A hive is a spawn source: parasites appear near it instead of anywhere unlit. It slowly turns nearby rock into infested rock (which drops biomass), and destroying it stops spawning on that asteroid. Big hives are guarded by a queen variant.

BUILDS ON

`MobSpawner` rules (unlit air next to natural blocks), `MobSenses`, `IMobBrain` states (forage, eat), loot tables, `LightManager`, data-only mobs (a new variant is just JSON), parasite sounds.

PLAYER EXPERIENCE

Clear objectives (clear the nest), a base-safety strategy (light it up and seal it), and a threat that grows the farther the player goes from home.

RISKS

Spreading infestation could cause many re-meshes, so it spreads at most 1 block per minute per hive and only near players (the same rule as parasite meals).

TECHNICAL NOTES (performance, network, save format)

The hive is a block type that registers in a `HiveRegistry` through `IEntityBlockListener`, and the spawner checks hives in range first. Infested blocks are new data-only block ids. Spreading happens through normal block edits on the server. Hive positions come deterministically from `AsteroidVoxelDataGenerator`, so untouched asteroids save nothing.

T2 Automated defense turrets

Effort M **Priority** P2 **Impact** ●●●●○

WHAT IT IS

A turret block with its own targeting: mobs, hostile drones and meteors, plus players of other crews when PvP is on. Each shot costs power, and ammo comes from an attached storage (laser turrets need power only).

BUILDS ON

`CannonBlock` (aim cone, `TurnSpeed`, `CannonTurretRenderer`, `CannonLineOfFire`), `MobSenses`-style targeting, authoritative `CombatNetwork` projectiles, the `Pvp` server rule, claim members as friends.

PLAYER EXPERIENCE

Base defense against T1, T3 and EV2.

RISKS

Turrets must not kill players by surprise: friend or foe comes from claim members and crews (M1), and players are targeted only with an explicit setting.

TECHNICAL NOTES (performance, network, save format)

Targets are chosen on the server at 4 Hz from `World.Physics` bodies and the `MobSystem` list, with line of sight through `Sector.Raycast`. Aim and shots reuse the cannon sync. Saves: `turret_data` (mode, target filter).

T3 Scavenger drones and pirate ships

Effort XL **Priority** P2 **Impact** ●●●●●

WHAT IT IS

AI-controlled ships built from prefab `.entity` files (as derelicts are) with real ship blocks. An AI pilot flies them through `AutoFlight` and aims their cannons. Behaviours: the **scavenger drone** steals drops and flees, the **raider** attacks valuable player-built entities and goes for thrusters first, and the **patrol** guards derelicts. Destroyed AI ships become salvage.

BUILDS ON

`ShipControlSystem` and `ConsoleCommand` (the AI writes the same commands a pilot does), `AutoFlight`, `ShipWeapons`, `StateMachine<TContext>` and `PathScheduler` from `Engine/AI`, the `MobSpawner` patterns (server only, despawn by distance), the Spacer NPC as crew to loot.

PLAYER EXPERIENCE

Cannons get a real target, and leaving the base becomes tense.

RISKS

Large effort. An AI that crashes into asteroids looks dumb. Many AI ships cost CPU on servers, so there is a cap per sector and idle ones despawn.

TECHNICAL NOTES (performance, network, save format)

A `ShipAiBrain` runs only on the server and feeds `ConsoleCommand`. Ships spawn from prefabs through `EntityCreationNetwork` and are not saved unless damaged or claimed (flag `ai_ship`). Sync goes through the existing `ShipStateMessage`. Depends on SH3 (autopilot) and X1 (prefabs).

E Economy and trade

There is no economy beyond crafting. These ideas use systems that already exist (tutorial tasks, the Spacer NPC, the analyzer) to add goals and exchange. (3 ideas: E1, E2, E3)

E1 Contracts board (reuses the tutorial engine)

Effort M **Priority** P1 **Impact** ●●●●●

WHAT IT IS

A contract terminal block (and later trade outposts) offers three rotating contracts built from templates in a GameSet `contracts.json`: deliver items, mine N of an ore, scan an asteroid, clear a parasite nest, fly to a waypoint, build and power a machine. Rewards are items, blueprint unlocks (E3) or reputation.

BUILDS ON

The tutorial runtime almost entirely: the task types (`have_item`, `destroy_block`, `process`, `craft`, `ship_travel`, `stat`, ...), `TutorialTaskFactory`, `PlayerActivity` events, `ITutorialContext`, the `TutorialUI` panel and toasts, and the rule that a reward is never an item the task asks for.

PLAYER EXPERIENCE

Endless short goals once the tutorial is over, and gentle guidance toward new biomes and systems.

RISKS

`TutorialRunner` assumes one tutorial for the local player. Extracting a general quest runner (with the tutorial as one kind of quest) is the main refactor risk. In multiplayer, contract state is per player.

TECHNICAL NOTES (performance, network, save format)

Progress is saved in the player JSON (`Contracts`, optional). The board is seeded from the world seed and the in-game day, so every player sees the same board. Delivery contracts take the items on the server through the container system. Tests in the style of `DefaultTutorialContentTests` check that every template maps to valid tasks and ids.

E2 Trade outposts and Spacer merchants

Effort L **Priority** P2 **Impact** ●●●●○

WHAT IT IS

Neutral outposts (structure prefabs, admin-protected or unbreakable) where Spacer NPCs buy and sell from `trade.json` price lists. Currency is an item (`credit_chip`), so it works with inventories, drops and death rules. Prices shift with what players sell (a simple supply curve per server). Admins can edit prices from the dashboard.

BUILDS ON

The Spacer NPC (`Storage`, `ContainerKey.ForObject`), structures (X1), `Unbreakable` blocks and `EntityProtection`, container sync with server validation, dashboard modules, `DisassemblyBalanceAudit`.

PLAYER EXPERIENCE

Surplus mining has a purpose, co-op players can specialise, and rare items become reachable.

RISKS

Economy exploits (craft cheap, sell high). An automatic audit test must ensure that no sell price exceeds the cheapest crafting cost, like `DefaultCrusherRecipesTests`.

TECHNICAL NOTES (performance, network, save format)

Trades are an `INetMessage` request that the server checks (stock, price, 6 m reach) before sending the result. Prices are kept in the world folder (`economy.json`, optional). Works headless.

E3 Research by analysis

Effort M **Priority** P2 **Impact** ●●●●○

WHAT IT IS

An optional GameSet rule: advanced blueprints start locked. Players unlock them by analysing samples (new ores, alien parts from derelicts) in the analyzer, or by disassembling found items (reverse engineering). Unlocks belong to the player or to the crew.

BUILDS ON

`AnalyzerScanner`, disassembly recipes, `crafting.json` blueprints (a new optional `Unlock` field), crafting categories, `PlayerActivity`, the tutorial's *Iron age* chapter.

PLAYER EXPERIENCE

Progress through discovery: exploration rewards knowledge, not only items.

RISKS

Existing worlds where players already know everything must keep working. A save without an unlock list means everything is unlocked, and the Default GameSet locks only new content.

TECHNICAL NOTES (performance, network, save format)

The unlock set lives in the player JSON (`Unlocks`, absent = all) and already reaches clients inside the joined player JSON. Check whether crafting is validated on the server; if not, the unlock check needs a server-side crafting request.

M Multiplayer: co-op and PvP

The server side is mature (claims, rules, dashboard, accounts). What is missing is the social layer: crews, shared progress and fair PvP. (3 ideas: M1, M2, M3)

M1 Crews

Effort M **Priority** P1 **Impact** ●●●●○

WHAT IT IS

Lightweight factions. Players create or join a crew. Crew members automatically get membership in each other's claims, share a crew waypoint list, can respawn at crew DNA reconstructors, see each other on radar with names, have a crew chat channel (`/c`) and get crate permissions.

BUILDS ON

`ClaimRegistry` members, `WaypointService` / `TagManager` (per-client today), `DnaReconstructorRegistry`, `ChatService`, `PlayerListNetwork`, `ProtectionRules.CanEdit`, the dashboard Players page.

PLAYER EXPERIENCE

Co-op feels like a team, and claim members no longer need to be added one by one.

RISKS

Permission complexity. Single player must stay unaffected.

TECHNICAL NOTES (performance, network, save format)

`crews.json` in the world folder (server). A `CrewStateMessage` is sent on join and on every change. Crew waypoints are stored on the server (`crew_waypoints.json`) and pushed when they change. `ProtectionRules` gets a crew check (a cheap set lookup). Dashboard module plus server guide update.

M2 Claim sieges (PvP servers)

Effort L **Priority** P3 **Impact** ●●●○○

WHAT IT IS

On PvP servers claims stop being absolute. During a raid window set by the server (or once the beacon's paid time runs out), attackers can break into a claimed base after destroying a shield generator powered by the base's own grid. Breach charges deal block damage that respects durability. Defenders get alerts in chat and on the dashboard.

BUILDS ON

`ClaimBeaconBlock` paid time and `ClaimSettings`, `ServerRules` (`ExplosionsRespectClaims`), `ProtectionRules.IsDamageProtected`, `ElectricNetworkManager`, explosives, the dashboard.

PLAYER EXPERIENCE

Meaningful PvP with preparation instead of offline griefing.

RISKS

Toxicity and griefing, so it is off by default and limited to scheduled windows.

TECHNICAL NOTES (performance, network, save format)

New `ClaimSettings` fields with defaults (`SiegeEnabled = false`, windows). The shield state is block state. All checks stay in `ProtectionRules` (one place).

M3 Leaderboards and server goals

Effort S **Priority** P2 **Impact** ●●●○○

WHAT IT IS

Leaderboards built from `PlayerStatistics` (distance, blocks, sectors visited, deaths by cause) in the pause menu and on the dashboard, plus server-wide goals ("deliver 10 000 titanium ingots to the gate") that unlock cosmetic rewards such as armor textures and sign colours.

BUILDS ON

`PlayerStatistics` / `PlayerStatsTracker`, dashboard modules, `ArmorRenderer` textures, sign colours, chat announcements.

PLAYER EXPERIENCE

Community goals on public servers, at low cost.

RISKS

Statistics are reported by the owner, so they can be cheated. Rank only stats the server can observe.

TECHNICAL NOTES (performance, network, save format)

The server aggregates the stats from player saves and sends them on request. A dashboard module plus an `app.js` page.

G Endgame goals

There is no long-term goal after titanium and ships. These are capstone projects that need every other system. (2 ideas: G1, G2)

G1 Belt Gate megaproject

Effort L **Priority** P2 **Impact** ●●●●●

WHAT IT IS

A long-term build: a huge multi-cell gate (16x16x4) assembled from modules. Each module needs late-game materials (obscurium, thulium lenses, uranium rods, microchips), and the gate needs a large, stable power supply. A charged gate opens a jump to a new belt region: a far sector band with harder biomes and new ores. In multiplayer it is a server-wide event. Ships and their riders jump together.

BUILDS ON

Multi-cell blocks (`Size` up to 16) and `MultiblockLayout`, `ElectricNetworkManager` (generators up to 1600), the sector system (8192 sectors per axis leaves plenty of room), `EntitySectorTracker`, `ShipRider`, `World.PreloadAroundPlayer` / `WorldLoadingScreen`.

PLAYER EXPERIENCE

A visible end goal that uses every system: mining, logistics, power, ships, and defense (events strike while the gate charges).

RISKS

Content cost of a new region, and moving large ships across sectors (streaming).

TECHNICAL NOTES (performance, network, save format)

The jump is `SetPose` into the far sector, preloading behind a loading screen. The new region is simply a distant biome band (X2), so no new generator is needed. Gate charge is block state plus a serializer (`gate_data`).

G2 Leviathan hive (boss)

Effort L **Priority** P3 **Impact** ●●●●○

WHAT IT IS

A rare giant asteroid (radius 200+) whose core holds a parasite queen and a hive network. Reaching it takes tunnelling, lights, oxygen outposts and defenses. Defeating the queen drops relics: unique blueprint unlocks or gate components.

BUILDS ON

T1 (hives), mob behaviour states with more HP and summoning, asteroid presets with cavities (`cheese`, `InternalCavities`), loot tables, E3 unlocks.

PLAYER EXPERIENCE

A dungeon-like expedition and a shared goal for a crew.

RISKS

Boss AI inside voxel tunnels; balancing for one player versus a group.

TECHNICAL NOTES (performance, network, save format)

The queen is a new `IMobBrain` registered in `MobBrainFactory`. The asteroid is a new preset with a guaranteed core cavity. Boss state (alive or dead) is saved in the asteroid NBT (`boss_defeated`, default false).

EV World events

Server-driven events make the belt feel alive and give players something to prepare for between sessions. (4 ideas: EV1, EV2, EV3, EV4)

EV1 World event director

Effort M **Priority** P2 **Impact** ●●●●○

WHAT IT IS

A server-side director schedules events from a GameSet `events.json` (weights, cooldowns, a player-progress threshold, biome conditions) and announces them (chat, HUD toast, sound). It is the framework for EV2-EV4, with an admin command `event start <id>` and a dashboard page.

BUILDS ON

`TickTaskManager`, `ServerRules` (a `world_events` rule through `ServerRulesService`), `ChatService`, dashboard modules, `MobSpawner`.

PLAYER EXPERIENCE

The belt feels alive, and stories emerge.

RISKS

Events that hit idle or AFK bases feel unfair, so they target only regions with active players.

TECHNICAL NOTES (performance, network, save format)

Events are `IWorldEvent` classes (start, update, end) registered in a factory, like `TutorialTaskFactory`. State is kept in the world folder (`world_events.json`, optional). Clients only receive `WorldEventMessage` (id, phase, position).

EV2 Meteor showers

Effort M **Priority** P2 **Impact** ●●●●○

WHAT IT IS

A swarm of small rock entities crosses a region at speed. They hit bases and ships (`ShipCollider` impulses plus block damage), can be shot down by turrets, and leave rare ore fragments behind.

BUILDS ON

A dynamic rock is just an entity with a `ShipBody`, `AsteroidVoxelDataGenerator` with a small radius, `ExplosionSystem` for impacts, projectile hit effects.

PLAYER EXPERIENCE

Players prepare their defenses, and it is spectacular.

RISKS

Collisions for many bodies cost CPU, so there are at most 20 rocks of at most 2 chunks each, despawned after they pass. Damage to claimed bases follows a rule.

TECHNICAL NOTES (performance, network, save format)

Rocks are simulated on the server and sent with `ShipStateMessage`. They are temporary entities and not saved unless they stop inside a base.

EV3 Solar flares

Effort S **Priority** P2 **Impact** ●●●○○

OVERLAP Needs S3.

WHAT IT IS

A warning 2 minutes ahead, then 60-90 s of flare: unshielded players outside take radiation (S3), electronics on exposed ships flicker (weaker thrusters), and radar range drops. Shelter is a sealed room or shielding blocks between the player and the flare direction.

BUILDS ON

S3 `RadiationField`, oxygen rooms, `VignetteController` and colour grading (screen tint), `RadarSettings`, the ambient audio channel.

PLAYER EXPERIENCE

A shared "get inside" moment that tests base design.

RISKS

Bad timing during long ship trips, so the warning time scales with distance to the nearest sealed room.

TECHNICAL NOTES (performance, network, save format)

Built on EV1 and S3. The flare direction is in `WorldEventMessage`. The client applies the effects to its own player.

EV4 Distress signals and comets

Effort M **Priority** P3 **Impact** ●●●○○

OVERLAP Needs X1.

WHAT IT IS

Distress signal: a derelict (X1) spawns some distance away with a radar tag and a timer; a Spacer survivor inside runs out of air unless rescued with a med kit or oxygen. **Comet:** a temporary icy entity with rare methane and thulium that crosses the sector for 20 minutes.

BUILDS ON

X1, radar tags, the Spacer NPC, the item gift system (`ItemGiftNetwork`), E1 contracts.

PLAYER EXPERIENCE

Spontaneous short adventures.

RISKS

Timed pressure in single player; the timer pauses while the game is paused.

TECHNICAL NOTES (performance, network, save format)

Event types inside the EV1 director. The comet is a temporary dynamic entity, removed from saves when it expires.

Prioritized roadmap

The order follows three rules: (1) first give players reasons to explore and to automate, because they multiply the value of the ships, machines and pipes that already exist; (2) build shared foundations once (structure prefabs, the quest runner, the event director, autopilot) and reuse them; (3) add pressure and threats only once players have the tools to answer them (turrets before raids, shelters before solar flares).



bar width = relative effort

Phase 1: A reason to leave the first asteroid (next 2 versions)

Cheap, high-leverage exploration and production features first. X3 and X2 work with today's content. X1 is the largest item in the phase and unlocks many later ideas.

1.	X3 Survey scanner pulse	S-M	P1
2.	X2 Resource biomes	M	P1
3.	T1 Parasite nests and infestation	M	P1
4.	A1 Auto-miner	M	P1
5.	S1 Fuel-burning generators and battery banks	M	P1
6.	SH3 Navigation computer: autopilot and cruise	M	P1
7.	M1 Crews	M	P1
8.	X1 Derelicts and structures	L	P1

Phase 2: Belt economy and engineering (following 2-3 versions)

E1 turns the tutorial engine into an endless goal system. Docking and propellant make ships part of the economy, and signals and turrets deepen bases.

1.	E1 Contracts board (reuses the tutorial engine)	M	P1
2.	SH1 Docking connectors and landing clamps	L	P1
3.	SH2 Propellant and fuel tanks	M	P2
4.	A2 Signal channels: sensors and actuators	L	P2
5.	A3 Logistics upgrades: sorter, stock signs, storage terminal	M	P2
6.	E3 Research by analysis	M	P2
7.	T2 Automated defense turrets	M	P2
8.	S4 Comfort and mood	S-M	P2
9.	M3 Leaderboards and server goals	S	P2

Phase 3: A living, dangerous belt (mid-term)

The event director plus real enemies. T3 and SH4 are the two XL items: schedule them after SH3, X1 and T2 are stable.

1.	EV1 World event director	M	P2
2.	EV2 Meteor showers	M	P2
3.	S3 Radiation	M	P2
4.	EV3 Solar flares	S	P2
5.	S2 Hull breaches and decompression	M	P2
6.	T3 Scavenger drones and pirate ships	XL	P2
7.	E2 Trade outposts and Spacer merchants	L	P2
8.	SH4 Ship damage model and splitting	XL	P2

Phase 4: Endgame (long-term)

Capstone goals that use every earlier system, and optional PvP depth for public servers.

1.	G1 Belt Gate megaproject	L	P2
2.	G2 Leviathan hive (boss)	L	P3
3.	M2 Claim sieges (PvP servers)	L	P3
4.	A4 Cargo launcher (mass driver)	L	P3
5.	SH5 Tractor beam and salvage cutter	M	P3
6.	EV4 Distress signals and comets	M	P3

Shared foundations and dependencies

Build first	Unlocks or simplifies
X1 Derelicts	E2 Trade outposts, T3 Pirates, EV4 Distress signals (shared prefab pipeline)
X2 Resource biomes	SH2 Propellant, SH3 Cruise, G1 Gate (destinations and the new region)
SH3 Autopilot	T3 Pirate AI (same flight controller)
S3 Radiation	EV3 Solar flares
EV1 Event director	EV2, EV3, EV4
T1 Parasite nests	G2 Leviathan hive
M1 Crews	T2 Turret friend or foe, M2 Sieges, E3 crew unlocks
E1 Contracts (quest runner)	E2 Trade outposts, EV4 rescue rewards

Recommended next three steps

1. **X3 Survey scanner pulse**: small and client-only, and it makes mining feel better at once. It also tests the new scanner tutorial step.
2. **A1 Auto-miner + S1 Fuel and batteries**, shipped together: the first real production loop (miner, pipes, furnace, fuel back into the reactor), the author's *automation* goal.
3. **X2 Resource biomes**, then **X1 Derelicts**: give the ships somewhere to go. Once `GenerationLoader.LoadStructures` works with prefab entities, it feeds trade outposts, pirates and distress signals later.

Every step ships with: JSON defaults in the `Default` GameSet, a server rule where the mechanic adds pressure, tests in `Spacebox.Tests` for the pure logic (scoring, fuel math, contract templates, prefab placement), a `DONE.txt` line, and an updated `spec.md` and tutorial.